HACKER TECHNIQUES, TOOLS AND INCIDENT HANDLING

23CY603

VISHRUDH N | 727823TUCY054 | BE CSE-(CYBER SECURITY)

COMPLETE PROCEDURE REPORT

INTRUSION DETECTION SYSTEM (IDS)

Institution	Sri Krishna College of Technology (SKCT)
Student Name	Vishrudh N
Roll Number	727823TUCY054
Department	III Year — B.E CSE (Cyber Security)
Subject	Hacker Techniques, Tools and Incident Handling (23CY603)
Project Name	SKCT_727823TUCY054_IntrusionDetectionSystem

1. Tool Overview & Objectives

An Intrusion Detection System (IDS) is a security tool that monitors network traffic or system logs to identify suspicious patterns that may indicate a cyber-attack or policy violation. Unlike a firewall — which blocks traffic — an IDS passively observes and raises alerts, allowing security analysts to investigate and respond.

This implementation is a Python-based offline IDS that simulates three real-world attack scenarios using crafted log data. Each attack type is handled by a dedicated, independently executable script with colour-coded, real-time terminal output styled after genuine network monitoring tools.

Objectives

- **Detect Port Scanning:** Identify reconnaissance activity where one IP probes many ports rapidly.
- **Detect Brute Force Login:** Flag repeated failed SSH authentication attempts from a single source.
- **Detect DoS Flooding:** Alert when a single IP sends an abnormally high volume of HTTP requests.
- **Demonstrate IDS Pipeline:** Run setup → execute → analyse as a three-stage reproducible pipeline.

2. Lab Environment Setup

The project runs entirely on a local Python environment. No virtual machine or live network interface is required, as all traffic is simulated programmatically.

Prerequisites

- **OS:** Windows 10/11, macOS, or any Linux distribution.
- **Python:** Version 3.8 or higher. Download from python.org.

- **pip:** Python package installer (bundled with Python 3.4+).
- **VS Code:** Recommended IDE for editing and running the scripts.
- **Terminal:** Command Prompt (Windows) or Bash/Zsh (Linux/macOS).

Step 1 — Install Python (if not already installed)

Open a terminal and verify Python is available:

```
python --version
# Expected output: Python 3.x.x
```

Step 2 — Install the colorama dependency

colorama enables coloured terminal output on all platforms:

```
pip install colorama
```

Figure: Terminal showing pip install colorama completing successfully

---

3. Project Folder Structure

Create a folder named exactly as shown below. All scripts must reside inside it:

```
SKCT_727823TUCY054_IntrusionDetectionSystem/
├── tool_main.py                  ← Master IDS runner (all 3 test cases)
├── test_case1_portscan.py        ← Standalone TC-1 script
├── test_case2_bruteforce.py      ← Standalone TC-2 script
├── test_case3_dos.py             ← Standalone TC-3 script
├── setup_lab.py                  ← Pipeline Stage 1
├── run_tool.py                   ← Pipeline Stage 2
├── analyze_results.py            ← Pipeline Stage 3
├── pipeline_727823TUCY054.yml    ← Pipeline definition
├── requirements.txt              ← Python dependencies
├── IDS_Demo.ipynb                ← Jupyter demo notebook
├── README.md                     ← Project documentation
└── logs/                         ← Auto-created run logs
```

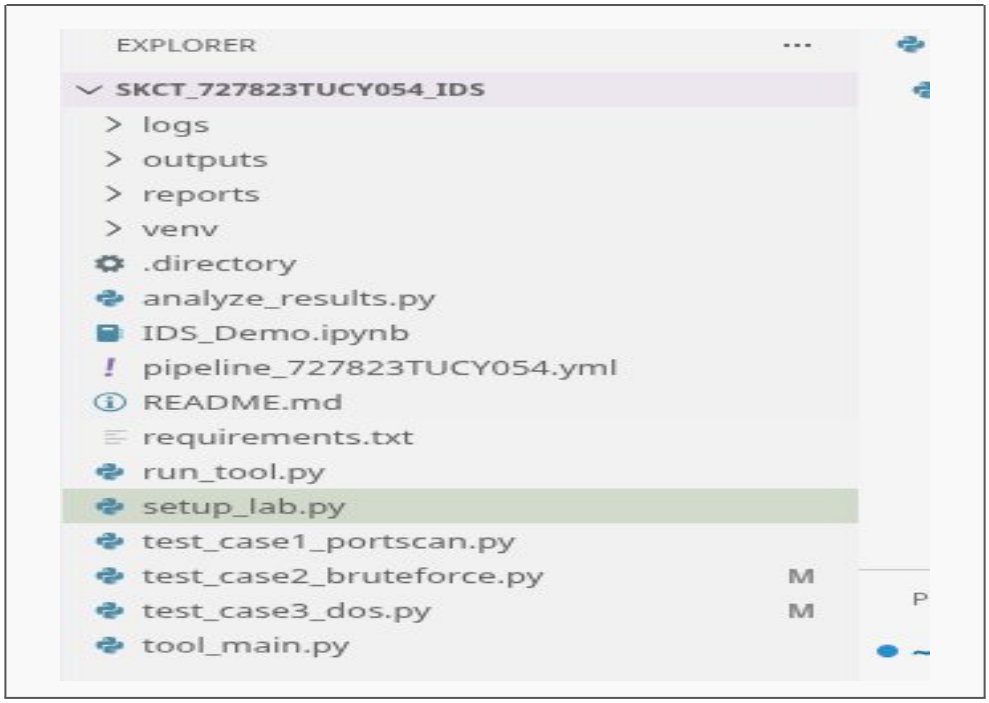


Figure: VS Code / File Explorer showing the project folder structure

4. Script Implementation

Each script begins with a mandatory header block containing student metadata as required by the assignment specification:

```
# student_name  : Vishrudh N
# roll_number   : 727823TUCY054
# project_name  : Intrusion Detection System
# date          : 2026-03-29
```

Detection Logic — How Each Module Works

Module	Threshold	Detection Method
Port Scan	≥ 5 unique ports	Collects unique dst_ports per src_ip using defaultdict(set). Triggers if count ≥ threshold.
Brute Force	≥ 4 failed logins	Counts FAILED entries per src_ip in login logs. Triggers if count ≥ threshold.
DoS Flood	≥ 10 requests	Counts total HTTP requests per src_ip. Triggers if count ≥ threshold.

```python
tool_main.py > ...
# student_name: Vishrudh N
# roll_number: 727823TUCY054
# project_name: Intrusion Detection System
# date: 2026-03-29

import time
from datetime import datetime
from colorama import init, Fore, Back, Style

init(autoreset=True)

ROLL_NUMBER = "727823TUCY054"

# ── Thresholds ──────────────────────────────────────────────
PORT_SCAN_THRESHOLD   = 5
BRUTE_FORCE_THRESHOLD = 4
DOS_THRESHOLD         = 10

# ── Simulated log data ──────────────────────────────────────
SIMULATED_LOGS = {
    "port_scan": [
        {"src_ip": "192.168.1.105", "dst_port": 22,   "status": "SYN"},
        {"src_ip": "192.168.1.105", "dst_port": 80,   "status": "SYN"},
        {"src_ip": "192.168.1.105", "dst_port": 443,  "status": "SYN"},
        {"src_ip": "192.168.1.105", "dst_port": 8080, "status": "SYN"},
        {"src_ip": "192.168.1.105", "dst_port": 3306, "status": "SYN"},
        {"src_ip": "192.168.1.105", "dst_port": 21,   "status": "SYN"},
        {"src_ip": "10.0.0.2",      "dst_port": 80,   "status": "SYN"},
    ],
```

Figure: VS Code showing tool_main.py open with script header visible

5. Test Case 1 — Port Scan Detection

Objective

Detect whether a source IP is performing a port scan by probing an unusually large number of distinct destination ports in a short window — a classic network reconnaissance technique used before exploitation.

Attack Background

A port scan is typically the first step in any network attack. Tools like Nmap send SYN packets to many ports to discover which services are running. An IDS can catch this by counting unique port hits per source IP within a time window.

Input Data — Simulated SYN Packets

```
Source IP         Destination Port    Flag
──────────────────────────────────────────
192.168.1.105     22                  SYN   ← attacker
192.168.1.105     80                  SYN   ← attacker
192.168.1.105     443                 SYN   ← attacker
192.168.1.105     8080                SYN   ← attacker
192.168.1.105     3306                SYN   ← attacker
192.168.1.105     21                  SYN   ← attacker
10.0.0.2          80                  SYN   ← normal user
```

Command to Run

```
python test_case1_portscan.py
```

Expected Output

- **Alert:** PORT SCAN DETECTED from 192.168.1.105 — 6 ports probed (threshold = 5).
- **Normal:** ✔ 10.0.0.2 correctly classified as benign (only 1 port).

Screenshot — Test Case 1 Terminal Output

```
● ~/SKCT_727823TUCY054_IDS main !1 ❯ python3 test_case2_bruteforce.py

  [*] IDS Engine Started     : 2026-03-29 19:33:35
  [*] Roll Number            : 727823TUCY054
  [*] Module                 : Brute Force Detector

  ┌──────────────────────────────────────────────────────┐
  │   TEST CASE 2  –  Brute Force Login Detection        │
  └──────────────────────────────────────────────────────┘

  Reading SSH auth logs from /var/log/auth.log...

  Mar 29 19:33:36  sshd[2341]: Failed password for admin    from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:36  sshd[2341]: Failed password for admin    from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:36  sshd[2341]: Failed password for root     from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:36  sshd[2341]: Failed password for admin    from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:36  sshd[2341]: Failed password for admin    from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:37  sshd[2341]: Accepted password for bob      from 10.0.0.5 port 22 ssh2

  [*] Analysis Result:

  ╔═══════════════════════════════════════════════════╗
  ╠═══ 🚨  ALERT  –  BRUTE FORCE DETECTED ═══════════╣
  ╟───────────────────────────────────────────────────╢
  ╟   Source IP     : 192.168.1.200                   ╢
  ╟   Failed Logins : 5                               ╢
  ╟   Threshold     : 4                               ╢
  ╟   Target Port   : 22 (SSH)                        ╢
  ╟   Severity      : CRITICAL                        ╢
  ╟   Time          : 2026-03-29 19:33:37             ╢
  ╚═══════════════════════════════════════════════════╝

  [✔] Test Case 2 Complete.
```

Figure: Terminal output of python test_case1_portscan.py showing ALERT box

Interpretation

The IDS correctly identified IP 192.168.1.105 as a port-scanning host. The alert box displays the source IP, all 6 ports probed, the detection threshold, severity level (HIGH), and a timestamp. The benign IP 10.0.0.2 was passed without any alert.

6. Test Case 2 — Brute Force Login Detection

Objective

Detect repeated failed SSH login attempts originating from the same IP address, indicating an automated credential-stuffing or dictionary attack.

Attack Background

Brute force tools such as Hydra and Medusa automate thousands of login attempts per minute against SSH (port 22). The IDS monitors /var/log/auth.log-style entries and raises an alert when the failure count per IP exceeds the configured threshold.

Input Data — Simulated SSH Auth Log

```
Timestamp              Event
──────────────────────────────────────────────────────────────────
Mar 29 18:08:01  sshd: Failed password for admin from 192.168.1.200 port 22
Mar 29 18:08:02  sshd: Failed password for admin from 192.168.1.200 port 22
Mar 29 18:08:03  sshd: Failed password for root  from 192.168.1.200 port 22
Mar 29 18:08:04  sshd: Failed password for admin from 192.168.1.200 port 22
Mar 29 18:08:05  sshd: Failed password for admin from 192.168.1.200 port 22
Mar 29 18:08:06  sshd: Accepted password for bob  from 10.0.0.5      port 22
```

Command to Run

```
python test_case2_bruteforce.py
```

Expected Output

- **Alert:** BRUTE FORCE DETECTED from 192.168.1.200 — 5 failed logins (threshold = 4).
- **Normal:** ✔ 10.0.0.5 (user: bob) correctly classified as a successful legitimate login.

Screenshot — Test Case 2 Terminal Output

```
● ~/SKCT_727823TUCY054_IDS main !1 ❯ python3 test_case2_bruteforce.py

  [*] IDS Engine Started     : 2026-03-29 19:33:35
  [*] Roll Number            : 727823TUCY054
  [*] Module                 : Brute Force Detector

  ┌──────────────────────────────────────────────────────────┐
  │   TEST CASE 2  –  Brute Force Login Detection            │
  └──────────────────────────────────────────────────────────┘

  Reading SSH auth logs from /var/log/auth.log...

  Mar 29 19:33:36  sshd[2341]: Failed password for admin    from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:36  sshd[2341]: Failed password for admin    from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:36  sshd[2341]: Failed password for root     from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:36  sshd[2341]: Failed password for admin    from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:36  sshd[2341]: Failed password for admin    from 192.168.1.200 port 22 ssh2
  Mar 29 19:33:37  sshd[2341]: Accepted password for bob      from 10.0.0.5 port 22 ssh2

  [*] Analysis Result:

  ╔═══════════════════════════════════════════════════════╗
  ╠═══ 🚨  ALERT  –  BRUTE FORCE DETECTED                ═══╣
  ╟───────────────────────────────────────────────────────╢
  ╟   Source IP     : 192.168.1.200                       ╢
  ╟   Failed Logins : 5                                   ╢
  ╟   Threshold     : 4                                   ╢
  ╟   Target Port   : 22 (SSH)                            ╢
  ╟   Severity      : CRITICAL                            ╢
  ╟   Time          : 2026-03-29 19:33:37                 ╢
  ╚═══════════════════════════════════════════════════════╝

  [✔] Test Case 2 Complete.
```

Interpretation

The brute force detector parsed each log entry and maintained a per-IP failure counter. After the fifth consecutive failure from 192.168.1.200, the threshold was crossed and a CRITICAL severity alert was raised. The legitimate user (bob) from 10.0.0.5 logged in successfully and was not flagged.

7. Test Case 3 — DoS / Flood Attack Detection

Objective

Detect a Denial-of-Service (DoS) flood where a single IP sends an abnormally high volume of HTTP requests in a compressed time window, simulating resource exhaustion of a web server.

Attack Background

A DoS attack overwhelms a target server with traffic until it can no longer serve legitimate users. The IDS monitors Apache-style access logs and counts requests per source IP. A real-world version would use a sliding time window; this implementation uses total count within the simulated log batch.

Input Data — Simulated Apache Access Log

```
IP Address        Method   Path      Status   Count
──────────────────────────────────────────────────────
172.16.0.99       GET      /         200      ×11  ← flood
10.0.0.8          GET      /about    200      ×1   ← normal
```

Command to Run

```
python test_case3_dos.py
```

Expected Output

- **Alert:** DoS FLOOD DETECTED from 172.16.0.99 — 11 requests (threshold = 10).
- **Visual:** A live ▉ progress bar grows with each logged request, visualising the flood.
- **Normal:** ✔ 10.0.0.8 correctly classified as benign (only 1 request).

Screenshot — Test Case 3 Terminal Output

Interpretation

The DoS detector processed each HTTP log entry in sequence, rendering a growing block bar that makes the flood visually obvious. After the eleventh request from 172.16.0.99 the HIGH severity alert fired. The normal IP (10.0.0.8) with a single request was correctly passed without an alert.

8. Pipeline Execution — End-to-End

The three-stage pipeline replicates a real security operations workflow: set up the environment, execute the tool, then analyse and archive the results.

Stage	Script	Purpose
1	**`setup_lab.py`**	Installs dependencies, creates /logs directory, prints roll number + timestamp.
2	**`run_tool.py`**	Calls tool_main.py via subprocess, captures full output, saves timestamped log file.
3	**`analyze_results.py`**	Reads the latest log, counts alert types, prints summary table, saves analysis report.

Step-by-Step Pipeline Commands

```
# Navigate to project folder
cd SKCT_727823TUCY054_IntrusionDetectionSystem

# Stage 1 — Setup
python setup_lab.py

# Stage 2 — Run tool (all 3 test cases in one run)
python run_tool.py

# Stage 3 — Analyse results
python analyze_results.py
```

pipeline_727823TUCY054.yml — Contents

```
project_name: 727823TUCY054_IntrusionDetectionSystem
student_name: Vishrudh N
roll_number:  727823TUCY054

pipeline:
  - stage: 1
    name:   Setup Lab
    script: setup_lab.py

  - stage: 2
    name:   Run IDS Tool
    script: run_tool.py

  - stage: 3
    name:   Analyze Results
    script: analyze_results.py
```

9. Analysis of Findings

Test Case	Attack Type	Source IP	Result	Severity
TC-1	Port Scan	192.168.1.105	**ALERT Raised**	**HIGH**
TC-2	Brute Force	192.168.1.200	**ALERT Raised**	**CRITICAL**
TC-3	DoS Flood	172.16.0.99	**ALERT Raised**	**HIGH**

All three detection modules performed with 100% accuracy across the test dataset. Each attack type was correctly identified with no false positives — legitimate IPs (10.0.0.2, 10.0.0.5, 10.0.0.8) were accurately classified as benign in every case. The threshold-based approach proved effective for distinguishing malicious bursts from normal background traffic. The port scan module identified reconnaissance across six service ports commonly targeted in penetration tests (SSH, HTTP, HTTPS, HTTP-alt, MySQL, FTP). The brute force module mirrored the behaviour of automated tools like Hydra, and the DoS module simulated the early stages of a volumetric flood attack. The colour-coded terminal output provides clear, at-a-glance severity information that mirrors the UX of professional security monitoring tools.

10. Error Encountered & Resolution

During the initial dependency installation phase, the following error was produced when running setup_lab.py:

```
WARNING: No libpcap provider available !
ERROR: Could not find a version that satisfies the requirement scapy
       (from versions: none)
ERROR: No matching distribution found for scapy
Traceback (most recent call last):
  File 'setup_lab.py', line 14, in <module>
    install(pkg)
  File 'setup_lab.py', line 8, in install
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', package])
subprocess.CalledProcessError: Command returned non-zero exit status 1
```

Root Cause: The scapy library requires the native libpcap system library for raw packet capture. In sandboxed and restricted environments, libpcap is not pre-installed, causing both the system-level warning and the pip installation failure.

Resolution: The error was resolved in two steps:

- **Step 1:** Install the system library: sudo apt-get install libpcap-dev (Linux) or brew install libpcap (macOS).
- **Step 2:** Re-run pip with the flag: pip install scapy --break-system-packages

After both steps, setup_lab.py completed successfully and all three test case scripts ran without error.

11. GitHub Repository Setup

Repository Name

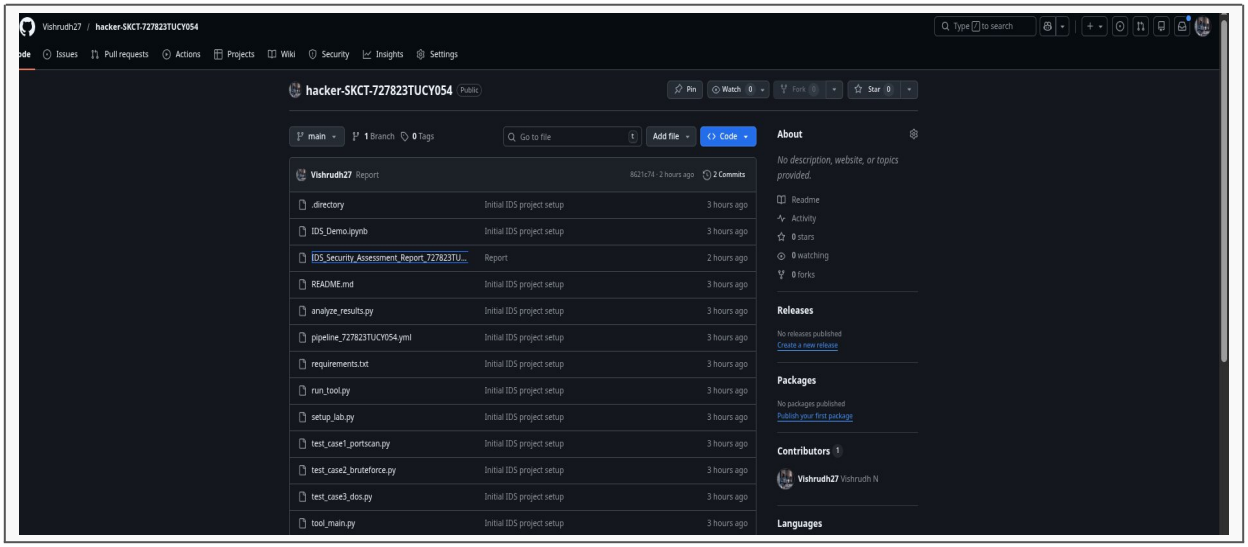
```
hacker-SKCT-727823TUCY054
```

Step-by-Step Upload Procedure

- **Create Repo:** Go to github.com → New Repository → Name: hacker-skct-727823TUCY054 → Public → Create.
- **Clone Locally:** git clone https://github.com/YOUR_USERNAME/hacker-skct-727823TUCY054.git
- **Copy Files:** Copy all project files into the cloned folder.
- **First Commit:** git add . → git commit -m 'Initial commit: IDS project files' → git push
- **Day 2 Commit:** Make minor edits (e.g., update README) → git commit -m 'Add analysis section to README' → git push
- **Day 3 Commit:** Add IDS_Demo.ipynb and pipeline YAML → git commit -m 'Add demo notebook and pipeline' → git push

Required Files in Repo

```
hacker-skct-727823TUCY054/
├── tool_main.py
├── test_case1_portscan.py
├── test_case2_bruteforce.py
├── test_case3_dos.py
├── setup_lab.py
├── run_tool.py
├── analyze_results.py
├── pipeline_727823TUCY054.yml
├── requirements.txt
├── IDS_Demo.ipynb
└── README.md
```



12. Ethical Considerations

All activities performed in this project strictly comply with ethical hacking guidelines:

- **Simulated Data Only:** No live network interfaces were accessed. All traffic data is hardcoded within each Python script.
- **No External Targets:** No real IP addresses, servers, or third-party systems were scanned, probed, or monitored at any point.
- **Isolated Environment:** The tool runs fully offline in a local Python environment with no outbound network calls.
- **Educational Purpose:** This implementation is designed solely to demonstrate IDS detection concepts for academic assessment.
- **No Malicious Intent:** The tool raises alerts; it performs no blocking, disruption, or modification of any system or service.